

SR Poultry Farm

Natural Naati Kodi - Raised Naturally

Healthy - Traditional - Farm Fresh



PRODUCTION-READY FULL-STACK WEB APPLICATION

Setup & Documentation Guide

Next.js + TypeScript + Tailwind - FastAPI + SQLAlchemy - PostgreSQL - JWT
- Docker Compose + Nginx

Customer Website	Premium landing, available birds, about, booking, how-to-buy
Admin Dashboard	Inventory, bookings, orders & billing, customers, revenue
Backend	FastAPI REST API, JWT auth, SQLAlchemy ORM, S3-ready storage
Deployment	docker compose up - AWS (ECS/RDS/S3/CloudFront) ready

SR Poultry Farm

Natural Naati Kodi - Raised Naturally

Healthy - Traditional - Farm Fresh



1. Overview

This application powers SR Poultry Farm, a premium natural country-chicken (Naati Kodi) brand. Customers browse available birds, read about the farm, and book a farm visit. Because birds are sold by live weight, purchases are recorded by the farm after the customer visits, selects a bird, and it is weighed - producing a final bill and payment record. An admin dashboard manages everything.

Technology stack (all open-source)

- **Frontend:** Next.js 14 (App Router), React, TypeScript, Tailwind CSS, Framer Motion - mobile-first, SEO-friendly, animated.
- **Backend:** FastAPI (Python), SQLAlchemy 2.0 ORM, Pydantic validation.
- **Database:** PostgreSQL 16.
- **Auth:** JWT bearer tokens + bcrypt password hashing.
- **Storage:** Local disk now, pluggable AWS S3 backend for later.
- **Infra:** Docker Compose (postgres, backend, frontend, nginx reverse proxy).

2. Prerequisites

- **Docker Desktop** with Docker Compose v2 (the only requirement for the quick start).
- For local (non-Docker) dev: Python 3.12+, Node.js 20+, and a local PostgreSQL instance.
- ~2 GB free disk for images and build artifacts.

3. Quick Start with Docker (recommended)

Step 1 - Copy environment files

SR Poultry Farm

Natural Naati Kodi - Raised Naturally

Healthy - Traditional - Farm Fresh



```
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env
cp .env.example .env
```

Step 2 - Edit backend/.env (important)

- Set a strong **SECRET_KEY** — generate with `openssl rand -hex 32`.
- Set **POSTGRES_PASSWORD** (use the same value in the root `.env`).
- Set **FIRST_ADMIN_EMAIL** / **FIRST_ADMIN_PASSWORD** for the seeded admin account.

Step 3 - Launch everything

```
docker compose up --build
```

On first boot the backend waits for PostgreSQL, creates all tables, and seeds a default admin plus sample Naati Kodi listings.

Step 4 - Open the app

http://localhost	Customer website (served via nginx)
http://localhost/admin/login	Admin dashboard login
http://localhost/docs	FastAPI interactive API documentation

Default admin (change in `backend/.env`): **admin@srpoultryfarm.com** / **Admin@12345**

Tip: with GNU Make installed, run `make setup` then `make up`.

4. Local Development (without Docker)

Backend

SR Poultry Farm

Natural Naati Kodi - Raised Naturally

Healthy - Traditional - Farm Fresh



```
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # set POSTGRES_HOST=localhost
python -m app.init_db
python -m app.seed
uvicorn app.main:app --reload --port 8000
```

Frontend

```
cd frontend
npm install
cp .env.example .env          # NEXT_PUBLIC_API_URL=http://localhost:8000
npm run dev                   # http://localhost:3000
```

5. Database Design

Table	Purpose
users	Admin / staff accounts (hashed passwords, roles)
chickens	Naati Kodi inventory (breed, weight range, price/kg, status)
chicken_images	Multiple images per bird
farm_visits	Visit bookings (date, slot, visitors, purpose) + status
customers	Buyer records
orders	Sales: actual weight x price/kg, total, status, payment status
payments	Payment records linked to orders

SR Poultry Farm

Natural Naati Kodi - Raised Naturally

Healthy - Traditional - Farm Fresh



6. Security

- JWT bearer authentication (python-jose) with configurable expiry and algorithm.
- bcrypt password hashing via passlib - plaintext passwords are never stored.
- Pydantic schema validation on every request body and query parameter.
- Admin-only endpoints protected by a reusable auth dependency.
- All secrets supplied through environment variables - nothing hard-coded.
- CORS restricted to explicitly configured origins.

7. Image Storage & AWS Migration

Storage is abstracted behind a single interface (app/storage). Local disk is the default and is served at /media. Switching to AWS S3 requires no code changes - only environment variables:

```
STORAGE_BACKEND=s3
AWS_S3_BUCKET=your-bucket
AWS_S3_REGION=ap-south-1
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_S3_PUBLIC_BASE_URL=https://cdn.yourdomain.com # optional CloudFront
```

Future production architecture on AWS

- Frontend & backend container images on **ECS Fargate** or **EKS**.
- Managed database on **RDS for PostgreSQL** (point POSTGRES_HOST at the RDS endpoint).
- Images on **S3** fronted by **CloudFront** CDN.
- TLS termination and routing via **Application Load Balancer** + ACM, DNS on **Route 53**.

8. Key API Endpoints

SR Poultry Farm

Natural Naati Kodi - Raised Naturally

Healthy - Traditional - Farm Fresh



Method	Path	Auth	Description
POST	/api/auth/login	-	Admin login (returns JWT)
GET	/api/chickens	-	List available birds
POST/PUT/DELETE	/api/chickens	Yes	Manage inventory
GET	/api/bookings/slots	-	Slot availability for a date
POST	/api/bookings	-	Create a visit booking
GET/PATCH	/api/bookings	Yes	Approve / reject / list
GET/POST	/api/orders	Yes	Orders & billing
POST	/api/orders/{id}/payments	Yes	Record a payment
GET	/api/dashboard/stats	Yes	Dashboard metrics
POST	/api/uploads/image	Yes	Upload an image

Full interactive documentation is available at </docs>.

9. Project Structure

```
sr-poultry-farm/  
  docker-compose.yml      postgres + backend + frontend + nginx  
  Makefile .env.example  
  nginx/nginx.conf        reverse proxy (/, /api, /media)  
  backend/  
    app/core/              config, database, security  
    app/models/            7 tables (see section 5)  
    app/schemas/          Pydantic models
```

SR Poultry Farm

Natural Naati Kodi - Raised Naturally

Healthy - Traditional - Farm Fresh



```
app/api/routes/      auth, chickens, bookings, orders,
                     customers, dashboard, uploads
app/storage/         local + S3 abstraction
main.py init_db.py seed.py Dockerfile requirements.txt
frontend/           Next.js (App Router) + TS + Tailwind
src/app/(site)/      public pages
src/app/admin/       admin login + dashboard
src/components/      src/lib/  src/types/
public/images/       sample farm + chicken SVGs
Dockerfile  package.json  tailwind.config.ts
```

(c) SR Poultry Farm - Raised Naturally - Healthy - Traditional - Farm Fresh